Operatori: calcolare, confrontare, assegnare

Lezione 4 — 1MAK, SST-INF-01c — Operatori matematici, di confronto, di assegnazione

02/10/2026

1 Ripasso: i tre tipi di contenitore

Nota

La volta scorsa abbiamo visto tre tipologie di **variabili**, cioè tre tipologie di **contenitori** in cui mettere dei dati:

- `char`: grande **8 bit**, cioè un byte; contiene numeri da un minimo di **0** a un massimo di **255**;
- `int`: minimo **16 bit**, cioè minimo 2 byte, ma dipende dall'**architettura della macchina**. Sui computer grossi, come quelli del laboratorio, che lavorano a **64 bit**, l'intero prende l'architettura della macchina. A noi interessano gli `int` da 16 bit, perché lavoreremo con **Arduino**, che usa `int` a 16 bit. Con una scatola da 16 bit si memorizzano numeri da -32.768 **a** $+32.767$;
- `float`: scatola da **32 bit**, cioè 4 byte, per i **numeri con la virgola**.

Trovate tutto nella **dispensa** della lezione precedente. La **tabella** scritta alla lavagna:

Tipo	Bit	Byte	Range detto in aula
`char`	8	1	$0 \to 255$
`int`	16	2	$-32.768 \to +32.767$
`float`	32	4	$-2\,\text{G} \to +2\,\text{G} \quad (-2 \cdot 10^9 \to +2 \cdot 10^9)$

Per non scrivere 9 zeri si usa la **notazione letterale** ("2 giga", 2G) oppure la **notazione scientifica** ($2 \cdot 10^9$), che va conosciuta perché la state facendo anche in matematica. Attenzione alle **due correzioni** qui sotto.

Nota

Il range del `char` in questo corso. In questo corso, e quindi anche nelle verifiche, per il `char` si usa il **range da 0 a 255**. Per completezza, **tre livelli** diversi. Lo **standard del linguaggio C non stabilisce** se il `char` ha il segno: **lascia la scelta** a chi realizza il compilatore. Il **compilatore** decide per la **piattaforma** per cui è fatto (e di solito la scelta si può cambiare con un'opzione). Per **Arduino**, la **documentazione ufficiale** indica per il `char` il range da -128 a $+127$, cioè **con segno**. Il range da 0 a 255 è sempre quello del tipo **senza segno**, **`unsigned char`** (su Arduino si chiama anche `byte`). In entrambi i casi le **combinazioni** sono 256.

Attenzione

Correzione sul range del `float`. In aula è stato detto che il `float` va "da circa meno **2 miliardi** a più 2 miliardi" (cioè da $-2 \cdot 10^9$ a $+2 \cdot 10^9$, in **notazione scientifica**). Quello è il **range** di un **intero** a 32 bit. Il `float` a 32 bit arriva **molto più in là**: circa $\pm 3{,}4 \cdot 10^{38}$, perché divide i suoi bit tra segno, **esponente e mantissa**, come la notazione scientifica. In cambio ha una precisione limitata, circa **7 cifre significative**. Il dettaglio è già nella dispensa A03.

Nota

Firmware o elettronica? Questa è una **lezione di firmware**. Come da calendario, il **lunedì** si fa **elettronica**, il **martedì e il venerdì** firmware, per riuscire ad andare avanti **in parallelo** col programma. In prima elettronica e firmware sono due cose diverse; in seconda cominciano a unirsi; dalla terza in poi saranno praticamente **un'unica cosa**, perché non si può parlare di elettronica senza parlare di firmware, e viceversa.

2 Cosa sono gli operatori

Definizione

Ora che sappiamo creare delle variabili, dobbiamo farle **interagire** tra loro: due contenitori lasciati lì da soli **non servono a niente**. Bisogna combinarli, cioè applicare delle **operazioni** che diano dei risultati. Alla fine dei conti il computer e l'Arduino sono macchine elettroniche che fanno una cosa sola: **calcoli**. Le operazioni le svolge la macchina, ma siamo noi a doverle **spiegare** come farle.
Gli **operatori** sono i simboli che permettono di **combinare** tra loro più contenitori (o più numeri).

Esempio

Se prendo i numeri 3 e 2 e li combino con l'operatore +, il risultato è **5**, perché è l'**operatore somma**. Se invece del più uso il meno, il risultato è diverso: **1**. Stessi numeri, **operatore diverso**, **risultato diverso**.

3 Gli operatori matematici

Definizione

Gli **operatori matematici** sono quelli studiati alle elementari, più uno nuovo. Sono **cinque**:

Simbolo	Operazione
+	somma
-	sottrazione
*	moltiplicazione (asterisco)
/	divisione (si chiama **slash**)
%	**modulo**: fornisce il resto della divisione tra interi

Esempio

Se in un programma scrivo `8 / 2`, vuol dire 8 diviso 2: il risultato è 4. Se scrivo `5 * 3`, è la **moltiplicazione**: fa 15.

Attenzione

Usate i simboli che la macchina si aspetta. Un allievo ha fatto notare che in matematica la moltiplicazione si scrive anche con il **puntino**. In matematica sì; in programmazione il puntino non si può usare, serve l'**asterisco**. Non potete scrivere `5.3` e aspettarvi che la macchina capisca che deve **moltiplicare**: per lei è il numero 5,3. La macchina **moltiplica solo** quando trova l'asterisco. Neanche la "x" va bene. Per la divisione non si usano i **due punti** né il simbolo $\div$, ma lo **slash** `/`, che sulla tastiera italiana sta sul tasto del 7 (`Shift + 7`).

3.1 L'operatore modulo

Definizione

Il simbolo `%` normalmente lo usate per la percentuale. In programmazione **non** ha niente a che vedere con la **percentuale**: anzi, il calcolo della percentuale come operatore **non esiste**, bisogna fare il conto **a mano**. Questo simbolo si chiama **modulo** e fornisce il **resto della divisione** tra interi.

Esempio

Gli esempi scritti alla lavagna.

- Quanto fa $5 \div 2$? In matematica 2 e mezzo. Ma con i soli **numeri interi**, cioè senza virgola, fa **2**, con il resto di **1**: $2 \times 2 = 4$, per arrivare a 5 manca 1. In programmazione `5 / 2` dà proprio `2`, e `5 % 2` dà `1`.
- $11 \div 3$: 3 per 3 fa 9, quindi fa **3** con il **resto** di **2**. `11 / 3` dà `3`, `11 % 3` dà `2`.
- $28 \div 5$: 5 per 5 fa 25, quindi fa **5** con il **resto** di **3**. `28 / 5` dà `5`, `28 % 5` dà `3`.

Attenzione

Il modulo **non** dà il risultato della divisione: dà **solo il resto**. Lo slash **non** dà il resto: dà solo il **risultato intero**. Se vi servono tutti e due, dovete scrivere **due righe di codice**: una con la divisione e una con il modulo. La macchina **non lo fa in automatico**.

```
int quoziente;
int resto;
// quoziente vale 5
quoziente = 28 / 5;
// resto vale 3
resto = 28 % 5;
```

Il modulo genera spesso un po' di **confusione**, per questo l'abbiamo ripetuto più volte. Tornerà utilissimo più avanti, per esempio per capire se un numero è **pari** o **dispari**: un numero è pari

se `numero % 2` **vale 0**.

Attenzione

La **divisione tra interi** **perde la parte decimale**. Il computer **non arrotonda**: **taglia via** tutto quello che c'è dopo la virgola. `5 / 2` dà 2, non 3. Per avere il risultato con la virgola servono variabili **`float`**.

4 Gli operatori di confronto

Definizione

Gli operatori matematici danno **sempre un risultato numerico**: $6 + 2$ fa 8, $9 - 5$ fa 4. È abbastanza ovvio, ma non era mai stato detto esplicitamente. Non tutti gli operatori fanno così: esiste una categoria di operatori che, invece di dare come risultato **un numero**, dà come risultato una **condizione di verità**, cioè **vero oppure falso**. Sono gli **operatori di confronto**. Sono **sei**:

In C	In matematica	Domanda
`>`	$>$	il primo è maggiore del secondo?
`<`	$<$	il primo è minore del secondo?
`>=`	$\geq$	il primo è maggiore **o uguale**?
`<=`	$\leq$	il primo è minore **o uguale**?
`==`	$=$	sono uguali? (**uguaglianza**)
`!=`	$\neq$	sono diversi? (**disuguaglianza**)

Esempio

Esempio alla lavagna.

```
int a;
int b;
a = 5;
b = 7;
a < b;
a > b;
```

`a < b` può dare un risultato numerico? No: può dare **solo vero o falso**. Allora, `a` è minore di `b`? In aula qualcuno ha risposto sì "perché la a viene prima della b". Attenzione: qui non stiamo chiedendo se la lettera `a` viene prima della `b` nell'alfabeto. `a` è una **variabile** che contiene 5, `b` contiene 7: la domanda è **5 è minore di 7?** Il confronto si fa sempre sul **contenuto** delle variabili. Quindi `a < b` è **vero**, mentre `a > b` è **falso**.

Esempio

E se `b` vale 5? Alla lavagna il docente cambia il valore di `b`:

```
int a;
int b;
```

```
a = 5;
b = 5;
a < b;
a > b;
a <= b;
```

`a` è minore di `b`? **No**: falso, 5 non è minore di 5. `a` è maggiore di `b`? **No**: falso anche questo. E `a` è minore **o almeno uguale** a `b`? **Sì**: vero, perché l'uguaglianza c'è. Da qui la necessità dei simboli "o uguale".
Durante queste domande un allievo ha risposto "le ho fatte, ma non me le ricordo": non è un problema, ma va superato, prendendosi il tempo a casa, la settimana successiva, per **ripassare** questi simboli.

Definizione

Maggiore o uguale, minore o uguale. Sulla tastiera **non esiste** il simbolo $\leq$ della matematica. In C si scrive **combinando due simboli**: `>=` equivale a $\geq$, `<=` equivale a $\leq$. Aggiungere l'uguale cambia la risposta sui **valori limite**: `5 < 5` è **falso** (5 non è minore di 5), ma `5 <= 5` è **vero**, perché almeno l'uguaglianza c'è.

5 L'operatore di assegnazione

Definizione

L'ultima categoria è quella che genera **più confusione** negli allievi di qualunque scuola, di ogni ordine e grado: per questo va fatta **con calma** e bene. Se in classe il professore di matematica scrive $a = b + 5$, sta scrivendo un'**equazione**: si domanda per quali valori di a è vera, cioè **confronta** quello che c'è a sinistra con quello che c'è a destra. È un simbolo di uguaglianza.
In C il simbolo `=` **non fa l'uguaglianza**. È l'**operatore di assegnazione**: **copia il valore** alla sua destra nella **variabile** alla sua sinistra. Non si sta chiedendo se sono uguali: si sta **spostando un'informazione** da un punto a un altro. Quindi l'**assegnazione** non dà mai un risultato vero o falso.

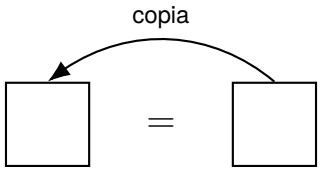


Figura 1: Lo schema disegnato alla lavagna: il valore della scatola di destra viene copiato in quella di sinistra.

Esempio

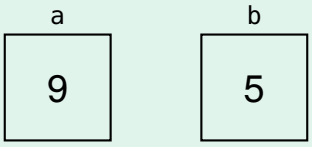
Quanto valgono `a` e `b` alla fine?

```
int a;
int b;
```

```
b = 9;
a = b;
b = 5;
```

In aula sono arrivate risposte di tutti i tipi: “a vale 14 perché 9 più 5”, “b vale 9 e a vale 11”, “b vale 7… no, 9… b era 5”. Molti allievi hanno risposto “b 9 e a 5”, cioè con i valori **scambiati**, e uno ha detto “a è uguale a b, sono due b”, come se le due variabili restassero legate. Alla fine la risposta giusta l’ha data un allievo. Dove sarebbe la somma? Per fare la **somma** serve il simbolo +, che qui non c’è. Perché proprio la somma e non la moltiplicazione o la divisione? La macchina **non si inventa le cose**: fa solo quello che c’è scritto.
Leggiamo il codice una riga alla volta, dall’alto verso il basso:

1. **creo** un contenitore che si chiama a;
2. **creo** un contenitore che si chiama b;
3. nel contenitore b **metto** il numero 9;
4. nel contenitore a metto lo **stesso contenuto** di b: in questo momento b vale 9, quindi in a va 9;
5. nel contenitore b metto 5: il 9 **sparisce** e al suo posto entra il 5.



Risultato: **a vale 9** e **b vale 5**, come nelle due scatole disegnate alla lavagna (non “b 9 e a 5”). Assegnare due volte b non è un problema: conta sempre **l’ultima assegnazione**. E se b cambia dopo, a **non cambia** (è l’errore di chi diceva “sono due b”): a = b **non crea un legame** tra le due variabili, dice solo “prendi il contenuto di b adesso e copialo in a”, poi basta.

Nota

In aula il docente ha insistito perché tutti rispondessero: “voglio interagire, non è possibile fare una lezione come un’orazione da cadavere”. Una **risposta sbagliata** non è un problema: si corregge. Il problema è **non provarci**.

6 Uguale singolo o doppio uguale?

Nota

Domanda di un allievo: “e i due uguali?”. Risposta: quello è il **confronto**. Alla lavagna, nella colonna degli **operatori di confronto**, il doppio uguale era stato scritto prima con l’etichetta “Confronto”, poi corretta in “**Uguaglianza**”, con sotto la “**Disuguaglianza**” !=.

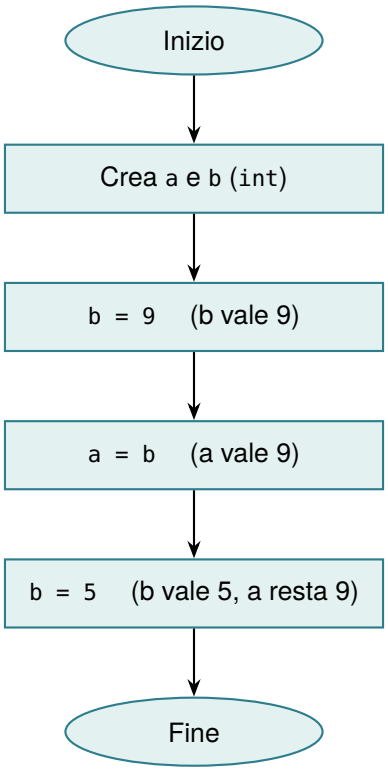


Figura 2: Diagramma di flusso dell'esempio: le istruzioni si eseguono una alla volta, dall'alto verso il basso.

Attenzione

L'**errore più comune** di chi inizia a programmare: confondere = con ==.

- = **singolo** serve ad **assegnare** un dato, cioè a copiarlo da un punto a un altro. È un'**azione**.
- == **doppio** serve a **confrontare**: chiede se due valori sono uguali e risponde vero o falso, senza modificare niente. È una **domanda**.

Ponetevi sempre il problema: quando devo copiare un dato, decidere quanto vale qualcosa, mi serve **l'uguale**; quando mi sto domandando se due valori sono uguali, devo usare il **doppio uguale**. È importante, perché sennò i vostri codici **non funzioneranno mai**. Attenzione anche a scrivere `5 = a`: a sinistra dell'assegnazione deve esserci **sempre una variabile**, perché **un numero non può contenere** un altro valore.

Esempio

Esempio alla lavagna.

```
int a;
int b;
a = 5;
b = 7;
a = b;
a == b;
```

`a = b` è un'**assegnazione**: prendo il contenuto di `b` e lo copio in `a`. Adesso `a` vale 7 e `b` vale 7. Poi `a == b` **chiede se sono uguali**: 7 è uguale a 7? **Vero**. Se togliessi la riga `a = b;`, il confronto sarebbe **falso**, perché chiederebbe se 5 è uguale a 7. Un conto è **copiare** il dato, un conto è **confrontare** due variabili o due numeri.

Esempio

La disuguaglianza. Con `a = 5` e `b = 7` (senza la riga `a = b`), scritto alla lavagna:

```
a != b;
a == b;
```

`a != b` chiede "a è diverso da b?": la **domanda contraria** dell'uguaglianza. È **vero**, perché 5 e 7 sono **numeri diversi**. `a == b` invece è **falso**. Si legge "di-su-gua-glian-za": **punto esclamativo** seguito da uguale.

7 Esercizi alla lavagna: vero o falso?

Esempio

Esercizio 1.

```
int a;
a = 11;
int b;
b = 3;
(a % b) == 2;
```

Alla lavagna l'esercizio è preceduto dalla scritta "V o F $\rightarrow$" (vero o falso?). È vero o falso? Prima si calcola quello che c'è **tra parentesi** (alla lavagna il docente ci ha messo sotto una graffa con scritto 2): `a % b` è il **resto** di $11 \div 3$, cioè **2** (non la divisione, che darebbe 3). Poi la domanda: 2 è uguale a 2? **Vero**.

Esempio

Esercizio 2.

```
int x;
int y;
int z;
x = 5;
y = 3;
z = x * y;
z > 10;
```

È vero o falso? `z` vale `x` per `y`, cioè $5 \times 3 = 15$: **l'asterisco** è la moltiplicazione. 15 è maggiore di 10? **Vero**.

8 Riepilogo degli operatori visti

Nota

La lavagna di fine lezione:

- **5 operatori matematici**: somma `+`, sottrazione `-`, moltiplicazione `*`, divisione `/`, modulo `%` (resto). Danno sempre un **numero**.
- **6 operatori di confronto**: maggiore `>`, minore `<`, maggiore uguale `>=`, minore uguale `<=`, uguaglianza `==`, disuguaglianza `!=`. Danno sempre **vero o falso**.
- **1 operatore di assegnazione**: `=`, che copia un dato da un punto a un altro (da destra a sinistra).

Con questi abbiamo **le basi** per cominciare a fare qualcosa. La prossima volta si accende il computer e si comincia con qualcosa di **più pratico**.

9 La prima verifica di firmware

Nota

Il docente ha chiesto alla classe se preferisse **poche verifiche con tanta roba** da studiare o **tante verifiche con poca roba**. Poi è stata concordata la **prima verifica di firmware**: **martedì 6 ottobre**, circa un'ora (c'erano tre settimane per studiare **tre facciate** di dispense). L'alternativa era il venerdì successivo, ma la classe ha preferito il martedì. Il docente va avanti con il programma e non fa un ripasso apposta: il **ripasso** tocca agli allievi. Alla domanda sul voto "da 0 a 100" in base al numero di domande, la risposta è stata che in una verifica conoscitiva le domande sono di **riconoscimento**. La lista scritta alla lavagna: **Binario** $\leftrightarrow$ **Decimale, Tipi variabili, Operatori**. È una verifica **conoscitiva**, non **operativa**, perché non abbiamo ancora imparato a "fare": domande a **risposta multipla**, del tipo "cos'è questo simbolo?" (è il minore uguale), "cosa fa il modulo?" (**calcola il resto**), "cosa fa il simbolo uguale?" (copia il valore alla sua destra nella variabile alla sua sinistra). Argomenti:

- conversione da **binario a decimale** e viceversa;
- **tipi di variabili**: che numeri può contenere un `int`? (da -32.768 a $+32.767$); per un numero con la virgola si usa `int` o `float`?;
- gli **operatori** di questa lezione.

Attenzione

Dispense e verifiche. Il docente aveva dimenticato di pubblicare la **dispensa** degli operatori (questa): chi non ha preso bene gli appunti la trova qui. Se una cosa è nella dispensa, **può essere nella verifica**, anche se a voce non è stata detta. Di solito le dispense contengono solo quello che viene detto a lezione, ma possono integrare qualcosa sfuggito a voce: **non basatevi** solo sui vostri appunti, leggete **tutta la dispensa**. Se nella dispensa ci fosse un errore e sbagliaste per colpa sua, **non verreste penalizzati**: le dispense vengono comunque controllate.

10 Cosa ricordare di questa lezione

- Gli **operatori** combinano variabili e numeri. In C si usano solo i **simboli** che la macchina si aspetta: asterisco per moltiplicare, slash per dividere.
- La **divisione tra interi** dà solo il risultato intero (`28 / 5` fa 5); il **modulo** `%` dà solo il resto (`28 % 5` fa 3). Per avere entrambi servono due righe.
- Gli operatori matematici danno un **numero**; gli operatori di confronto danno **vero o falso**.
- Il confronto si fa sul **contenuto** delle variabili, non sul loro **nome**.
- `>=` e `<=` si scrivono con due simboli e includono il **valore limite**.
- `=` **assegna** (copia da destra a sinistra), `==` **confronta**. Non confonderli **mai**.
- Il codice si legge **una riga alla volta**, dall'alto verso il basso: conta **l'ultima assegnazione**, e `a = b` **non lega** le due variabili per sempre.